Matching Cut on Bipartite Graphs of Radius Three

Research draft · awaiting human expert review
15 September 2026

Abstract
A matching cut is a nontrivial vertex bipartition in which each vertex has at most one neighbor across the cut. For bipartite graphs, the published bounded-radius classification leaves a gap between polynomial-time solvability at radius two and NP-completeness at radius four. We give a deterministic Karp reduction from the radius-four case to the radius-three case, together with polynomial-time recovery from every target witness. Two monochromatic bicliques and constant-size binary and selector blocks encode signed exact-one constraints. Equal biclique colors force a constant coloring; opposite colors enforce the Boolean constraints. The construction has $O(n^3)$ vertices and edges and $O(n^3 \log(n+1))$ encoding size. It establishes NP-completeness for bipartite graphs of radius at most three, subject to expert validation of this draft.

1. Introduction
Matching Cut asks whether a graph can be partitioned into two nonempty parts so that the crossing edges form a matching. A small graph radius limits distances without bounding degrees, so hardness at small radius requires transmitting logical choices through short paths without creating extra crossing neighbors.

Lucke [1, Table 1 and Section 5, Open Problem 1] records polynomial-time solvability on bipartite graphs of radius at most two and NP-completeness at radius at most four. The radius-three case is explicitly open there. Le and Le [2, Section 3, Theorem 2] prove hardness for bipartite graphs of diameter four, which also supplies the radius-four source problem.

Theorem 1. There is a deterministic polynomial-time many-one reduction from Matching Cut on connected simple bipartite graphs of radius at most four to the same problem at radius at most three. For an input with n vertices, the output has $O(n^3)$ vertices and edges. Every valid target witness can be transformed deterministically into a source witness in polynomial time. Consequently, Matching Cut on connected bipartite graphs of radius at most three is NP-complete.

The construction uses two disjoint copies of $K_{3,3}$ joined by one edge. These copies supply reference colors without imposing a perfect-matching condition. Private blocks either collapse to the common reference color or encode Boolean choices when the references differ. Their attachment edges are monochromatic, allowing arbitrarily many occurrences of a variable while keeping every vertex within distance three of one distinguished vertex.

2. Definitions and reference colors
For a graph $G = (V, E)$, let $N_G(v)$ be the neighbors of v. A matching-cut witness is a nonconstant map $x : V \to \{0, 1\}$ satisfying

$$\forall v \in V, \quad |\{u \in N_G(v) : x(u) \neq x(v)\}| \leq 1. \tag{1}$$

Call a vertex *saturated* when it already has one opposite-colored neighbor in a specified subgraph. Every further incident edge in a valid extension must be monochromatic. Radius is the minimum, over vertices, of the maximum distance to another vertex.

Let L_S and L_T be the languages in Theorem 1. Their graph encodings list an integer order and distinct ordered endpoint pairs, with vertex labels starting at zero. Strings that do not encode the required

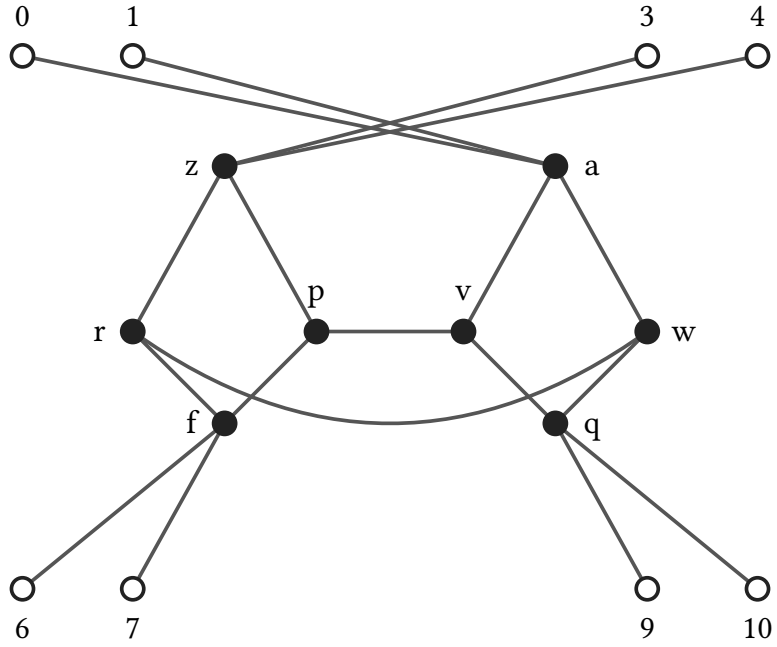


Figure 1: Binary interface. Open numbered circles are shared reservoir vertices; filled circles are private vertices. The occurrence ports are v, w. Reservoir-internal edges and the edge 06 are omitted here and shown in Figure 2. Fill distinguishes vertex roles, not witness colors. The curved rw edge has no intermediate vertex.

connected simple bipartite graph lie outside the respective language. Write $R_S(G, x)$ and $R_T(H, y)$ for their graph-domain and witness predicates.

Create disjoint reservoirs U and W, each inducing $K_{3,3}$, with sides

$$U : \{0, 1, 2\}/\{3, 4, 5\}, \quad W : \{6, 7, 8\}/\{9, 10, 11\}, \tag{2}$$

and add the edge 06. A slash here separates the two sides of a biclique, not cut colors. The global bipartition reverses the orientation of W.

Lemma 2 (reservoir rigidity). In any coloring satisfying Equation 1, each embedded $K_{3,3}$ is monochromatic. A vertex adjacent to two distinct vertices of one reservoir has its color.

Proof. If a side of the biclique contains both colors, its majority color occurs at least twice. Every vertex on the other side must have that color to avoid two opposite neighbors. The minority vertex on the first side then has three opposite neighbors, a contradiction. Each side is therefore monochromatic, and the two sides cannot differ. Restricting a full coloring to a subgraph preserves the upper bound in Equation 1. The last claim follows from the two specified neighbors. □

3. Constant-size interfaces

All private vertices introduced below are distinct between blocks. Reservoir labels always refer to the same twelve shared vertices. Complementing every color preserves Equation 1, so the proofs normalize the color of U to zero.

3.1. Binary interface

A binary block has private vertices f, p, r, z, a, v, w, q. Its edges are

$$\{f6, f7, z3, z4, a0, a1, q9, q10\}, \tag{3}$$

$$\{fp, fr, zp, zr, av, aw, qv, qw, pv, rw\}. \tag{4}$$

Juxtaposition denotes an unordered edge. Figure 1 displays these eighteen edges; it also identifies the two occurrence ports.

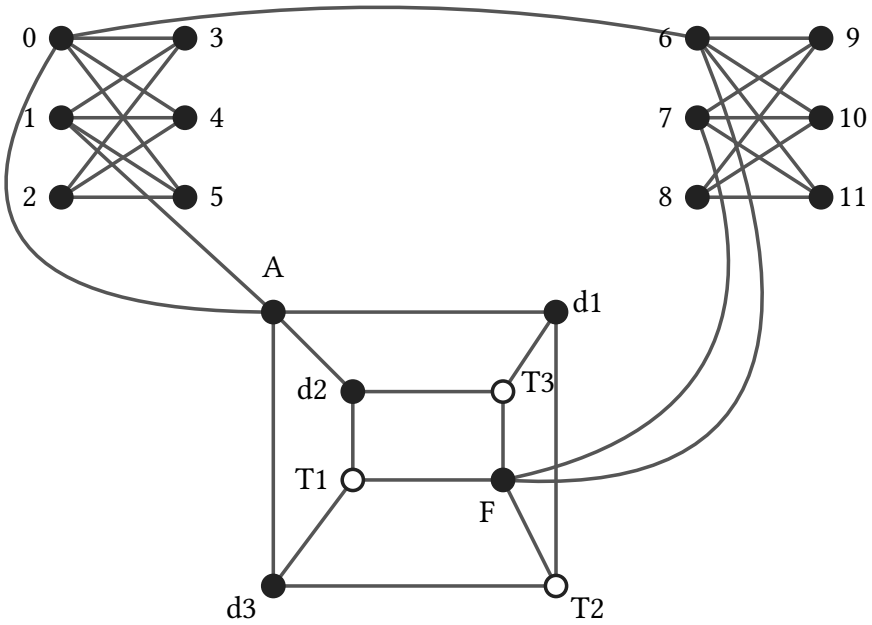


Figure 2: The two reservoirs and one selector, with all twenty vertices and thirty-five edges shown. Open circles are the high ports T_1, T_2, T_3; fill does not encode colors. The edge 06 joins the reservoirs. Two-neighbor attachments force A to the color of U and F to the color of W. When those colors differ, exactly one d_i differs from A, and each T_i has the complementary color to d_i.

Lemma 3 (binary relation). If the reservoirs have colors zero and one respectively, the block has exactly two internal extensions, with

$$f = q = 1, \quad z = a = 0, \quad p = v = b, \quad r = w = 1 - b, \quad b \in \{0, 1\}. \tag{5}$$

The four vertices p, r, v, w are saturated. If the reservoirs have the same color, every private vertex has that color. These implications hold with arbitrary additional incident edges.

Proof. Lemma 2 forces the four anchor colors. Each of p, r sees both $f = 1$ and $z = 0$; each of v, w sees both $a = 0$ and $q = 1$. They are saturated, so the edges pv and rw imply equality of their endpoint colors. The bounds at a and q force v and w to differ. Either choice of b satisfies all internal bounds, including those at the anchors. If the reference colors agree, a private occurrence vertex of the other color would have two opposite anchor neighbors. All private vertices must then agree with the references. Additional edges can only restrict these relations. □

3.2. Exact-one selector

A selector has private vertices $F, A, d_1, d_2, d_3, T_1, T_2, T_3$. Add the edges

$$\{F6, F7, A0, A1\} \cup \{Ad_i, FT_i : i \in \{1, 2, 3\}\} \tag{6}$$

$$\cup \left\{d_i T_j : i, j \in \{1, 2, 3\}, i \neq j\right\}. \tag{7}$$

The vertices T_i are high ports. Figure 2 shows a selector with both complete reservoirs.

Lemma 4 (selector relation). With reservoir colors zero and one, respectively, the selector permits exactly the three assignments in which the low bits d_i are one-hot and $T_i = 1 - d_i$. Every high port is saturated. Equal reservoir colors force all selector vertices to that common color. These implications remain valid under further attachments.

Proof. Lemma 2 gives $A = 0$ and $F = 1$. The bound at A permits at most one low bit equal to one. If all low bits were zero, each high bit would have to be zero, since it has two low neighbors. This would give F three opposite neighbors. There is therefore a unique index k with $d_k = 1$.

The edge $d_k A$ saturates d_k, forcing $T_j = 1$ for $j \neq k$. Each $d_j = 0$ with $j \neq k$ sees one of these high bits and must have $T_k = 0$. The vertex T_k has its only internal opposite at F. Each other high port has its only opposite at the remaining zero low bit. Directly counting neighbors verifies all other bounds and every one-hot extension.

If both reference colors are zero and some $d_i = 1$, its edge to A forces the other two high bits to one, violating the bound at $F = 0$. Hence all low bits are zero. A high bit equal to one would then see three zero neighbors. Complementation handles common reference color one. □

4. Boolean encoding and the complete rule

4.1. Encoding a source cut

Introduce a Boolean variable x_v for each source vertex. For each vertex b and unordered pair of distinct neighbors a, c, impose

$$(x_a \vee \neg x_b \vee x_c) \wedge (\neg x_a \vee x_b \vee \neg x_c). \tag{8}$$

These clauses prohibit precisely the two assignments in which both neighbors differ from b. Add the disjunction of all x_v and the disjunction of all $\neg x_v$ to require a nonconstant coloring. The resulting formula expresses Equation 1 exactly.

Convert each disjunction with more than three literals by emitting $(\ell_1 \vee \ell_2 \vee z)$ with a fresh variable z and continuing on $(\neg z, \ell_3, \ldots, \ell_k)$. Repetition finishes the chain. A satisfying original clause extends by setting each chain bit to indicate that the processed prefix is false; eliminating the chain bits by resolution proves the converse. Repeat the final literal of a shorter clause until its length is three.

Write $E(u, v, w)$ for the constraint that exactly one of its three signed Boolean literals is true. Replace each three-literal disjunction by fresh selector variables s_i and slack variables t_i with constraints

$$E(s_1, s_2, s_3), \quad E(s_i, \neg \ell_i, t_i) \quad (i = 1, 2, 3). \tag{9}$$

The latter equations give $t_i = \ell_i - s_i$, so $s_i \leq \ell_i$. A selected literal must be true. Conversely, select the first true literal and set each slack bit by this equation. Fresh variables make the replacements independent. These Boolean conversions are standard; the contribution concerns their graph realization at radius three.

4.2. Forward map and recovery

For a legal source of order at least two, first build its signed exact-one instance. Introduce one binary block per encoded variable and one selector per exact-one clause, using the common reservoirs. For each clause position i, connect its high port T_i to the variable port w for a positive literal and to the port v for a negative literal. Both choices connect to the complement of the signed literal's value. Distinct clause positions have distinct high ports, even for repeated literals.

Order the encoded variables with the original source variables first. Binary block i receives labels $12 + 8i$ through $19 + 8i$ in the order used in Lemma 3. The recovery map on a promised valid target witness is

$$g(G, y)_i = y_{13+8i} \oplus y_0, \quad 0 \leq i < |V(G)|. \tag{10}$$

It requires only the source and the supplied witness, and works in a fresh process. The singleton source maps to $K_{3,3}$, a legal NO target. Invalid source strings map to the invalid target encoding `{}`.

Lemma 5 (equivalence and universal recovery). For the stated maps,

$$G \in L_S \leftrightarrow f(G) \in L_T, \quad R_T(f(G), y) \Longrightarrow R_S(G, g(G, y)). \tag{11}$$

Proof. A source witness extends to a satisfying signed exact-one assignment by the Boolean conversions. Color U zero and W one and extend every block using Lemmas 3 and 4. Each literal attachment is monochromatic because both its endpoints have the complementary literal value. All reservoir attach-

ments are also monochromatic. The sole crossing edge at reservoir vertices is 06, so sharing reservoirs or reusing occurrence ports creates no extra crossing neighbors. This gives a nonconstant target witness.

Conversely, consider any nonconstant target witness. Lemma 2 makes both reservoirs monochromatic. If their colors agreed, Lemmas 3 and 4 would force every private vertex to agree, contradicting nonconstancy. Normalize their colors to zero and one. Every literal attachment is monochromatic because its endpoints are internally saturated. Thus its low selector bit equals its signed literal value, and Lemma 4 enforces each exact-one clause. The Boolean conversions imply the original neighbor-pair restrictions and nontriviality clauses. The recovery map in Equation 10 projects exactly these source bits. This argument applies to every target witness and proves the NO direction as well. The exceptional singleton and invalid encodings satisfy the claimed equivalence by their definitions. □

5. Structure and complexity

The first side of a bipartition consists of reservoir vertices $0, 1, 2, 9, 10, 11$, binary vertices f, z, v, w, and selector vertices F, d_1, d_2, d_3. Every listed edge crosses this bipartition. Distinct private labels and high ports prevent parallel edges. Every block connects to the joined reservoirs, so the output is connected.

Vertex zero has distance at most three to every output vertex. Inside U, the other vertices on its side have distance two and the opposite side has distance one. In W, vertex six has distance one, vertices nine through eleven have distance two, and vertices seven and eight have distance three. The private vertices have the following upper bounds, witnessed by the construction edges:

Block	Distance 1	Distance 2	Distance 3
Binary	a	f, z, v, w	p, r, q
Selector	A	F, d_1, d_2, d_3	T_1, T_2, T_3

These paths exist on both YES and NO inputs. Literal edges cannot increase their lengths. The exceptional singleton output has radius two.

For source order $n \geq 2$, define the neighbor-pair count B, the number of chain variables Z, and the number of three-literal clauses C by

$$B = \sum_{b \in V(G)} \binom{\deg(b)}{2}, \quad Z = 2\max(n-3, 0), \tag{12}$$

$$C = 2B + 2\max(n-2, 1). \tag{13}$$

There are $N = n + Z + 6C$ encoded variables and $K = 4C$ exact-one clauses. Counting the fixed interfaces gives

$$|V(f(G))| = 12 + 8N + 8K, \quad |E(f(G))| = 19 + 18N + 19K. \tag{14}$$

Since $B = O(n^3)$, the graph has the claimed order and size, and uses $O(n^3 \log(n+1))$ bits with integer endpoint labels.

The executable orders all vertices, neighbor pairs, auxiliaries and clauses deterministically. It uses only the Python standard library. Domain testing checks the schema, numeric types, edge uniqueness, connectivity, bipartiteness and radius, using breadth-first search for the graph properties. Before allocation it rejects $n > 2m + 1$, where m is the number of listed edges; connected inputs satisfy this necessary condition. Thus an invalid enormous order cannot force allocation unrelated to input length.

For encoded input length L, a conservative worst-case bound for parsing, validation, integer operations, construction and serialization is $O((L+1)^5)$ bit operations. Allocated graph order is $O(L)$; charging even worst-case set comparisons polynomially avoids an expected-time assumption. Decimal JSON integer syntax is polynomially equivalent to binary integer encoding. Recovery reads its full input and performs n indexed selections, so it is polynomial in the combined source and target-witness encoding

length. Neither map calls a solver or uses random choices. Membership in NP follows by checking Equation 1 and the target graph domain, completing Theorem 1.

6. Classification and scope

The construction closes the bounded-radius gap stated in [1], placing the hardness boundary immediately above radius two. Biclique rigidity and the Boolean conversions are established techniques; the claimed contribution is their radius-three composition with universal witness recovery. Perfect Matching Cut requires exactly one opposite neighbor and is a different predicate. No conclusion about that variant or Disconnected Perfect Matching is needed here.

The literature search of 15 September 2026 inspected [1] and [2] and found no later resolution of this exact case. This supports novelty within the checked coverage; unindexed, unpublished, inaccessible and overlooked work remain possible. The draft has passed a native independent agent review. Human expert assessment of correctness, novelty and significance remains outstanding.

References

[1] Felicia Lucke. *Matching cut and variants on bipartite graphs of bounded radius and diameter.* Theoretical Computer Science 1052 (2025), 115429. Published version arXiv:2501.08735. The published Table 1 and introduction were inspected; the precise Open Problem 1 location refers to Section 5 of the readable preprint.

[2] Hoàng-Oanh Le and Van Bang Le. *A complexity dichotomy for Matching Cut in (bipartite) graphs of fixed diameter.* Theoretical Computer Science 770 (2019), 69–78. Primary preprint, Section 3, Lemmas 4–6 and Theorem 2.

A. Verification and reproducibility

Finite checks test the executable; the lemmas establish correctness for arbitrary input sizes. The prepared oracle was cross-checked against exhaustive witnesses on eleven connected bipartite atlas graphs through five vertices, for both source and target predicates. Three intentionally incorrect programs were rejected. The candidate then passed eight prepared cases with twenty recovery calls. These include the singleton, an edge, a three-vertex path, a four-cycle, two complete bipartite NO cases, a radius-four path and the cube.

A separately implemented author-side verifier passed eighteen cases, fourteen YES and four NO, with fifty-six recovery calls. Its largest output had 3,628 vertices and 8,331 edges. The suite includes the same atlas bound, four specified six-vertex graphs and three fixed radius-four trees. These families overlap the prepared suite; their counts are executions, not distinct-graph totals.

The independent reviewer imported none of the candidate or author checkers. Its exhaustive local checks considered all 256 private colorings for each block with equal and opposite reference colors. Full executable checks on an edge, a three-vertex path and $K_{2,3}$ returned YES, YES and NO. Four actual recovery calls passed. Six bounded queries excluding bad projections or nonconstant equal-reservoir cuts returned UNSAT; four were nonvacuous on YES outputs. Six malformed or out-of-domain inputs and the singleton case were also checked.

The first prepared run used an arithmetic encoding and was manually interrupted while inside Z3. It returned `unknown` and is excluded from passing evidence. Replacing the same at-most-one predicate by Z3's native Boolean cardinality constraint required one checker change, a repeated oracle self-test and the successful complete rerun. There were no candidate algorithm repairs and no configured timeouts. Details and the preserved earlier checker are in the round record.

From the campaign directory, Python 3 with installed `z3-solver` and `networkx` reproduces the checks. The author oracle's recorded Z3 version was 5.1.0. The reduction itself has no third-party dependency. Typst and a PDF renderer are needed only for this document.

```
python3 work/check.py --self-test
python3 work/check.py --candidate work/algorithm.py
python3 work/verify.py --candidate work/algorithm.py
python3 reviews/complete-rule/check.py
```

The review checker resolves its campaign path from its own location. The forward command reads a graph JSON object; recovery reads the original source and one valid target witness:

```
python3 work/algorithm.py < source.json > target.json
python3 work/algorithm.py --extract < recovery-input.json
```

`source.json` has exactly the fields `n` and `edges`; each edge is a distinct pair of integers in increasing order. `recovery-input.json` has fields `source` and `target_solution`, with a nonconstant list of integer bits as the target witness. The checks solve actual target outputs before calling recovery, including complemented witnesses.

The executable and proof are `work/algorithm.py` and `work/proof.md`. Prepared and additional results are under `rounds/001/`; independent evidence is `reviews/complete-rule/{check.py,checks.json,review.md}`. Exact figure generation is `rounds/001/figures.py`. Compile from `work/` with `typst compile manuscript.typ manuscript.pdf`. The source includes vector figures whose displayed adjacency was checked against the construction.

Novelty queries included the phrases "Matching Cut" with "radius three", "radius 3", "radius-three", "bipartite", "2026", and the equivalent "decomposable" terminology. The review records source locations and access limits: later attempts to download the entire published PDF returned HTTP 403, so an unread published conclusion is not attributed an open-problem number.